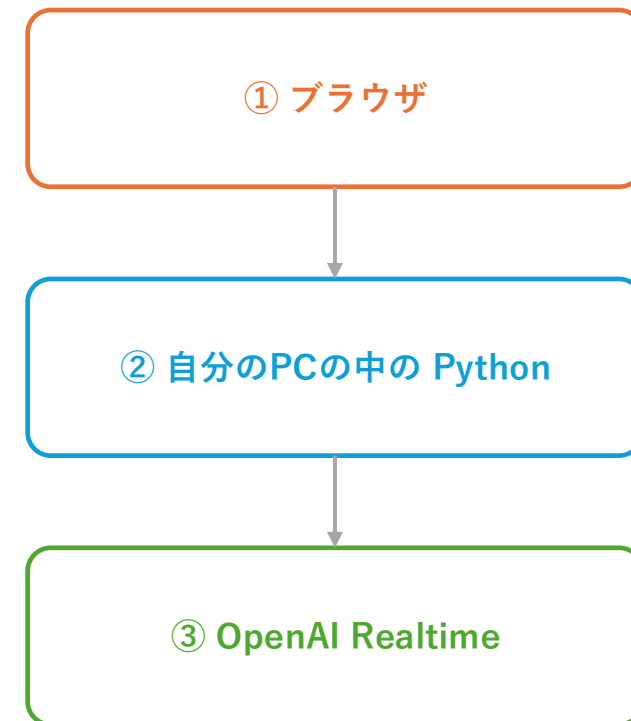


AI エージェントに 個性を与えよう！

名古屋大学教育学部附属高等学校 SSH 講座

2026年7月30日(木)・31日(金) 10:30 – 14:30



2日間の流れ

1日目 (7/30)

自分のPCに開発環境をつくる

- ① PowerShell とフォルダ移動
- ② Python 3.12 を入れる (PATH)
- ③ VSCode と拡張機能
- ④ git clone — 教材を迎える
- ⑤ venv / ⑥ pip install

12:30 昼休憩

- ⑦ .env と APIキー

⑧ 起動して、初めての会話

短いループの儀式

エラーからの復帰

2日目 (7/31)

自分のAIに個性を与える

個性① — 声と間合い

個性② — プロンプト

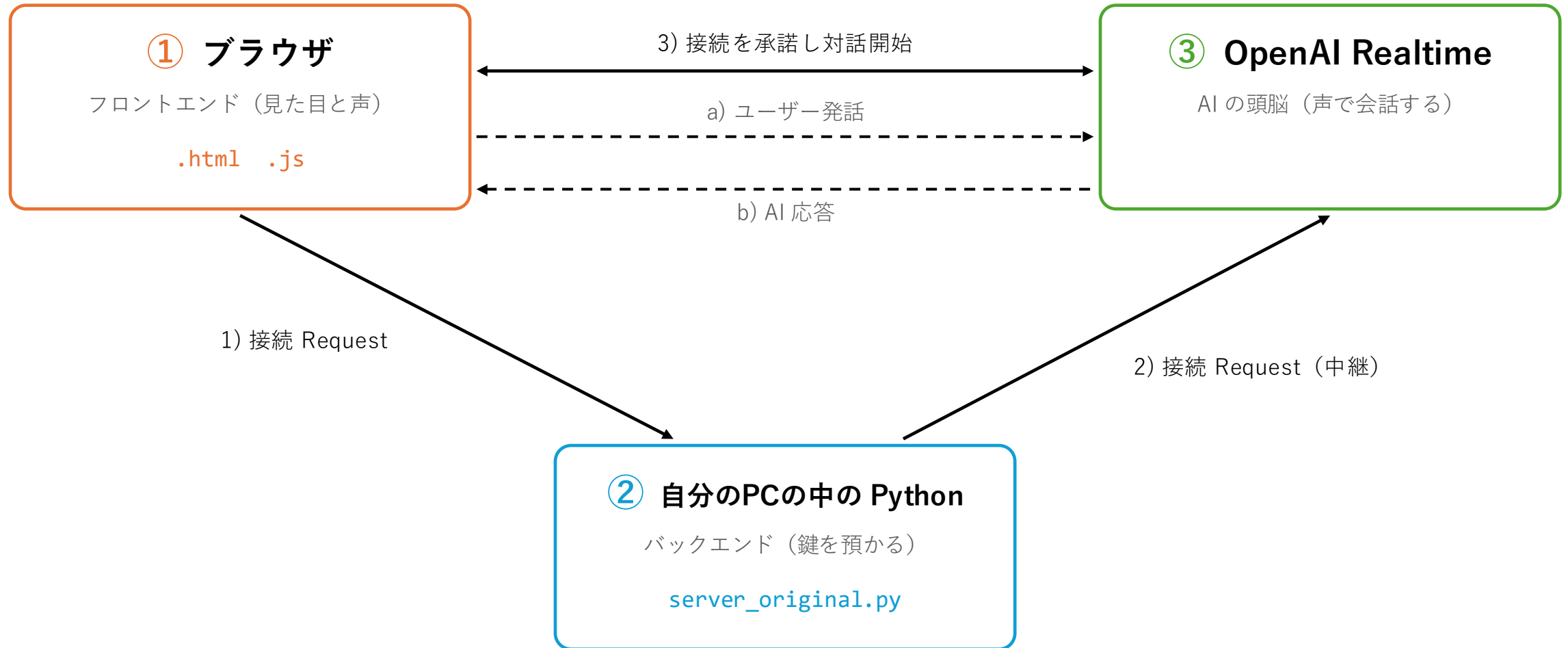
自由制作

12:30 昼休憩

自由制作

作ったやつ共有

基本システム



環境構築の全体像

	やること	打つコマンド（Windows）	なぜ必要か
①	PowerShell を開く	<code>pwd / ls / cd</code>	自分が「今どこにいるか」を知る
②	Python 3.12 を入れる	<code>py --version</code>	AI と話すプログラムを動かす本体
③	VSCode + 拡張3つ	日本語 / Python / Live Server	コードを書く道具を整える
④	教材を持ってくる	<code>git clone https://github.com/...</code>	世界に公開されたコードを複製する
⑤	venv をつくって入る	<code>py -m venv venv → venv\Scripts\activate</code>	このプロジェクト専用の小部屋を作る
⑥	部品を入れる	<code>py -m pip install -r requirements.txt</code>	動かすのに必要なライブラリを集める
⑦	APIキーを置く	<code>.env</code> に <code>OPENAI_API_KEY=sk-...</code> の1行	OpenAI に金払ってると示す鍵
⑧	起動して画面を開く	<code>py -m uvicorn server_original:app --reload</code>	ここで初めて AI と声で話せる

コマンドはスライドからコピーしない。リポジトリの中の `commands.txt` からコピーすること（スライドの文字は見た目が同じでも別の記号のことがある）。

環境構築 ① PowerShell を開く

GUIとCUIは別世界ではない。同じフォルダを、別の角度から見ているだけ。

1 スタートボタンを押して「powershell」と入力し、Windows PowerShell を開く

黒い（または青い）画面が出る。これがコンピュータへの直通の窓口。

2 今どこにいるかを表示する

```
pwd
```

Path のところに出るのが「カレントディレクトリ」 = いま自分がいるフォルダ。

3 そこに何があるかを表示する

```
ls
```

エクスプローラを横に並べて、同じものが見えていることを確認する。

4 デスクトップへ移動する

```
cd $HOME\Desktop
```

フォルダをダブルクリックするのと同じこと。プロンプトの表示が変わる。

5 一つ上のフォルダへ戻る

```
cd ..
```

「..」は「一つ上」という意味の特別な名前。

✓ うまくいったサイン

```
PS C:\Users\あなた> cd $HOME\Desktop
```

```
PS C:\Users\あなた\Desktop>
```

行の先頭の表示が、いま自分がいる場所が変わる。

⚠ つまづいたら

フォルダ名に日本語やスペースが入っている

引用符で囲む → `cd "マイ フォルダ"`

打ち間違いが多い

Tab キーで名前を自動補完、↑ キーで前のコマンドを呼び戻せる

PATH とは何か

コマンド名だけ渡された OS が、その実体を探しに行く「フォルダの順番リスト」

あなたが打つ

py

OS が上から順に探す

...\SSH-Realtime-Agent\venv\Scripts ← 見つければここで終わり

C:\Users\あなた\AppData\Local\Programs\Python\Python312

C:\Windows\System32

だから、こうなる

- 同じ py でも、人によって別のものが動く
- 「入れたのに動かない」の正体は、たいてい **PATH に載っていない**
- **venv を activate する** = この順番リストの先頭に、自分の小部屋を割り込ませること

確かめてみよう

where python

実体はどこにある？

echo %env:Path

どの順番で探している？

わざと打ち間違えてみる → **pythn --version**

command not found は「壊れた」ではなく
「探したけれど見つからなかった」と言っているだけ。

環境構築 ② Python 3.12 を入れる

AI と話すプログラムの本体。バージョンが古いと、2日目の教材だけが動かなくなる。

1 まず、入っているかどうかを確認する

```
py --version
```

バージョンが出れば次へ。何も出ない／エラーなら、この先へ進む。

2 インストーラを起動する（USB または python.org）

python-3.12.x-amd64.exe をダブルクリック。

3 最初の画面の一番下 —— 「Add python.exe to PATH」 に必ずチェックを入れる

★ここを忘れると、あとで py も pip も「見つからない」になる。今日いちばん大事なチェックボックス。

4 「Install Now」 を押して待つ（2～3分）

終わったら Close で閉じる。

5 PowerShell をいったん閉じて、開き直す

PATH は起動したときに読み込まれる。開き直さないと反映されない。

6 もう一度、確かめる

```
py --version
```

Python 3.12.x と出れば成功。

✓ うまくいったサイン

```
PS C:\Users\あなた> py --version
```

Python 3.12.10

3.12 以上であることを目で確認する。

⚠ つまづいたら

python と打つと Microsoft Store が開く

Store 版のエイリアス。公式インストーラを入れ、以降は py コマンドを使う

3.9 や 3.8 が出る

古いバージョンが先に見つかる。3.12 を入れて PowerShell を開き直す

「Add python.exe to PATH」を忘れた

設定 → アプリ → Python → 変更 → Modify から追加し直す

環境構築 ③ VSCode と拡張機能

エディタは「メモ帳 + 工具箱」。魔法の箱ではなく、道具に道具を足していく仕組み。

1 インストーラを起動する（User Installer でよい）

User Installer なら管理者権限が要らない。

2 「PATH への追加」にチェックを入れて、インストール

これで PowerShell から `code .` と打てるようになる。

3 VSCode を起動し、左側の四角が4つ並んだアイコン（拡張機能）を押す

上の検索欄に名前を打ち込んで探す。似た名前が並ぶので、発行元を必ず見ること。

4 拡張機能を3つ入れる（それぞれ Install を押す）

Japanese Language Pack / Python / Live Server

日本語 = Microsoft、Python = Microsoft、Live Server = Ritwick Dey。
Python を入れた前後で .py の色が変わるのを見る。Live Server は後で html を http:// で開くために使う。

5 統合ターミナルを開く

Ctrl + @

さっきの PowerShell が、VSCode の中に入っているだけ。同じもの。

✓ うまくいったサイン

VSCode の中の下半分にターミナルが出る
その中に PS C:\> が表示される
.py を開くと文字に色が付いている

⚠ つまづいたら

管理者権限で弾かれる

System Installer ではなく User Installer を選び直す

拡張機能が似た名前ばかりで選べない

発行元（publisher）を見る。Python は Microsoft、Live Server は Ritwick Dey

環境構築 ④ 教材を自分のPCに迎える

GitHub は世界中の人が設計図を置いている場所。clone は「履歴ごと複製する」こと。

1 まず、ブラウザで教材のページをしてみる

```
https://github.com/TeradaLab-Agents/SSH-Realtime-Agent
```

誰でも見られる。これが「公開されている」ということ。

2 どこに置くかを自分で決めて、そこへ移動する

```
cd $HOME\Desktop
```

① で覚えた cd がここで効いてくる。置き場所は自分が決める。

3 複製する

```
git clone https://github.com/TeradaLab-Agents/SSH-Realtime-Agent
```

数十秒。Cloning into ... と出て終われば成功。

4 できたフォルダに入って、中身を見る

```
cd SSH-Realtime-Agent    その後    ls
```

出てきたファイルが、さっきの3箱図のどれに当たるかを1つずつ trying みる。

5 VSCode でこのフォルダを開く

```
code .
```

最後のピリオドは「今いるフォルダ」という意味。

✓ うまくいったサイン

pharmacy_agent.html	← ① ブラウザ
pharmacy_agent.js	← ① ブラウザ
server_original.py	← ② Python
server_function-calling.py	
requirements.txt	← 部品リスト

⚠ つまづいたら

git というコマンドが無いと言われる

緑の Code ボタン → Download ZIP。展開すると同じ名前のフォルダが二重になることがあるので中を確認

venv（仮想環境）とは何か

このプロジェクト専用の部屋の的なもの。壊れたらフォルダごと捨ててやり直せる。

使わないと…

- PCの中に Python の部屋が1つしかない
- 去年の別プロジェクトのライブラリと、今日のライブラリが競合する
- 壊れたとき、どこまで戻せばいいかわからない

使うと

- このフォルダの中だけに部品が入る
- 他のプロジェクトに一切影響しない
- 壊れたら **venv** フォルダを消して作り直すだけ

つくる → 入る → 出る

```
py -m venv venv          # 小部屋をつくる
venv\Scripts\activate    # 中に入る
```

(venv) と出たら、部屋の中にいる

```
deactivate               # 外に出る
```

環境構築 ⑤ venv をつくって入る

打つ場所に注意。SSH-Realtime-Agent フォルダの中で打つこと。

1 いま正しい場所にいるか確かめる

```
pwd
```

末尾が ¥SSH-Realtime-Agent になっているか。違えば cd で移動する。

2 小部屋をつくる

```
py -m venv venv
```

10～20秒。venv という名前のフォルダができる。

3 小部屋に入る

```
venv\Scripts\activate
```

行の先頭に (venv) が付く。ここが最重要の確認ポイント。

4 本当に切り替わったか確かめる

```
where python
```

さっきと違う場所 (venv¥Scripts の中) を指していれば成功。

5 (終わるとき) 小部屋から出る

```
deactivate
```

(venv) が消える。今日は出なくてよい。

✓ うまくいったサイン

```
(venv) PS C:\Users\...\SSH-Realtime-Agent>
```

行の先頭に (venv) が付いていること。ここを見ずに進むと必ず後で詰まる。

⚠ つまづいたら

「このシステムではスクリプトの実行が無効になっているため…」

venv¥Scripts¥activate.bat を使う。または Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass を先に打つ (今の画面だけに効く設定)

(venv) が付かないまま進んでしまった

気づいた時点で activate し直して、pip install からやり直す

環境構築 ⑥ 部品（ライブラリ）を入れる

requirements.txt は買い物リスト。pip install はそれを一括で買ってくる作業。

1 (venv) が付いているか、もう一度だけ確認する

付いていなければ `venv¥Scripts¥activate`。

2 わざと、まだ入っていない状態で起動してみる

```
py -m uvicorn server_original:app --reload
```

ModuleNotFoundError: No module named 'fastapi' と出る。この文を声に出して読む。

3 買い物リストの中身を見る

VSCode で requirements.txt を開く。6〜7行しか書かれていない。

4 リストどおりに部品を集めてくる

```
py -m pip install -r requirements.txt
```

20秒ほど。最後に Successfully installed ... と出る。

5 何が入ったか数えてみる

```
py -m pip list
```

6行のリストから、実際には何個入ったかを見る。部品も部品を必要とする。

✓ うまくいったサイン

Successfully installed fastapi-0.1xx.x

httpx-0.2x.x pydantic-2.x.x uvicorn-0.3x.x

python-dotenv-1.x.x websockets-x.x

⚠ つまづいたら

ModuleNotFoundError がまだ出る

(venv) が付いていない状態で入れた可能性。activate してから入れ直す

pip というコマンドが無いと言われる

py -m pip の形で打つ。こうすれば「今使っている Python」に確実に紐づく

エラーメッセージの読み方

エラーは何が足りないかを教えてくれる。まず最後の行を読む。

command not found

そんな名前のコマンドは、PATH のどこにも無かった

どうする

打ち間違い？ / まだ入れていない？ / PATH に載っていない？

ModuleNotFoundError

Python は動いたが、その部品が入っていない

どうする

(venv) は出ている？ → `py -m pip install -r requirements.txt`

Address already in use

その番号の窓口は、すでに誰かが使っている

どうする

前のサーバが生きている。Ctrl+C で止めてから起動し直す

401 Unauthorized

鍵が違う、または鍵が読めていない

どうする

.env の場所と名前を確認 (.env.txt になっていない？)

ファイル名と行番号が書いてあれば、そこを開く。それだけで9割は解決する。

環境構築 ⑦ .env と APIキー

なぜ ② バックエンドの箱が必要なのか？

もし鍵をブラウザ側を書いたら

```
// pharmacy_agent.js  
const KEY = "sk-proj-abc123...";
```

→ ページを見た人は全員、その鍵を読める。使い放題になる。

だから、こうする

```
# server_original.py L88  
os.getenv("OPENAI_API_KEY")
```

→ 本物の鍵は ② の中だけ。ブラウザに渡すのは使い捨ての Ephemeral Key (L118)。

1 VSCode の左のファイル一覧で、いちばん上の「新しいファイル」アイコンを押す

エクスプローラから作ると .env.txt になりやすい。VSCode から作るのが確実。

2 ファイル名を .env とだけ入力する（先頭のピリオドを忘れない）

SSH-Realtime-Agent フォルダの直下に置くこと。

3 配られた鍵を、次の1行の形で貼り付けて保存する（Ctrl + S）

```
OPENAI_API_KEY=sk-proj-XXXXXXXXXXXXXXXXXX
```

4 server_original.py の L88 を開いて、この1行を読んでいることを確かめる

os.getenv("OPENAI_API_KEY") ← ここがさっき書いた .env を読んでいる。

APIキーはクレジットカードと同じ。ネットに自分のキーを公開すると他人に利用される恐れがある。講座が終わったら回収します。

環境構築 ⑧ 起動して、画面を開く

ここまでの全部（PATH・venv・pip・.env）が1つにつながって「動く」瞬間。

1 (venv) が付いていることを確認して、サーバを起動する

```
py -m uvicorn server_original:app --reload
```

Application startup complete. が出るまで待つ。この画面は開いたままにしておく。

2 証拠その1 —— ブラウザで、サーバが活着ていることを確かめる

```
http://127.0.0.1:8000/docs
```

/api/realtime-proxy が並んでいれば、②の箱は活着ている。

3 証拠その2 —— ターミナルにログが流れることを見る

さっき自分が入れた uvicorn が喋っている。「動いた気がする」で済ませない。

4 VSCode で pharmacy_agent.html を右クリック → Open with Live Server

http://127.0.0.1:5500/... でブラウザが開く。file:// で開いてはいけない。

5 マイクの使用を「許可」する

ブラウザの許可と、Windows の 設定 → プライバシー → マイク の両方が要る。

6 「おはなしをはじめる」を押して、話しかける

★ ここが1日目のゴール。終わるときは Ctrl + C。

pharmacy_agent.js の L77 に BACKEND_URL が書いてある。①の箱が②の箱を名指ししている配線の実物。

✓ うまくいったサイン

INFO: Uvicorn running on http://127.0.0.1:8000

INFO: Application startup complete.

そのうえで、画面のアバターが表示され、声が返ってくること。

⚠ つまづいたら

Address already in use

netstat -ano | findstr:8000 で番号を調べ、taskkill /PID 番号 /F で止める

画面が「準備しています…」から進まない

アバターの画像が読めていない。F12 → Network タブで赤い行を見る

「マイクが使用できません」と出る

Chrome を使う（Firefox では必ず失敗する）

ここまでのチェックリスト

自分ひとりでもう一度この状態にできれば、環境構築を身につけたことになる。

- ☐ ② `py --version` で 3.12 以上が出る
- ☐ ③ VSCode に 日本語 / Python / Live Server の3つが入っている
- ☐ ④ デスクトップに SSH-Realtime-Agent フォルダがある
- ☐ ⑤ そのフォルダの中に `venv` フォルダがある
- ☐ ⑤ 行の先頭に `(venv)` を出せる
- ☐ ⑥ `py -m pip list` に `fastapi` と `uvicorn` がある
- ☐ ⑦ フォルダ直下に `.env` がある (`.env.txt` ではない)
- ☐ ⑧ `http://127.0.0.1:8000/docs` が開ける
- ☐ ⑧ 自分のPCで、AI と声で会話できた

明日の朝、これだけ打つ

```
cd $HOME\Desktop\SSH-Realtime-Agent
```

```
venv\Scripts\activate
```

```
py -m uvicorn server_original:app --reload
```

そのあと Live Server で html を開く

フロントエンド

フロントエンド pharmacy_agent.html / .js

UI 状態管理

画面の見た目とボタン動作を“状態”ごとに切り替える

1. agentLoading

「エージェントを準備しています」

2. ready

「おはなしをはじめる 🎤」

3. connecting

「AI に接続しています」

4. active

メインの対話画面（録音中）

5. muted

メインの対話画面（ミュート中）

WebRTC

マイク音声ストリームを取得し OpenAI と“直接つながる”

1. マイク取得

ブラウザと OS の両方で許可が要る

2. Offer / Answer の交換

通信の決まりを決定する

3. 音声再生開始

Answer を承諾し通信開始

DataChannel

OpenAI と双方向でやり取りし、UI／アバターを動かす

① ユーザー発話テキスト化

② AI 応答テキスト化

③ 応答開始（口パク開始）

④ 応答終了（口パク終了）

Function Calling 関連

⑤ 関数呼び出し開始

⑥ 引数チャンク

⑦ 関数呼び出し完了

バックエンド

バックエンド `server_original.py`

ブラウザから SDP Offer を受け取る → OpenAI Realtime API に転送 → 返ってきた SDP Answer をブラウザに返す

Ephemeral Key を取りに行くときの、主なパラメータ

パラメータ	説明	場所 (<code>server_original.py</code>)
<code>model</code>	利用する会話モデル名	L93 <code>gpt-realtime-2.1-mini</code>
<code>instructions</code>	システムプロンプト (AI の人格を決める)	L94 ← 個性の主戦場
<code>audio.input.transcription</code>	音声 → テキスト変換 (whisper-1 / language)	L96-100
<code>audio.input.turn_detection</code>	発話検出。threshold = 敏感さ、 <code>silence_duration_ms</code> = 間	L101-106
<code>audio.output.voice</code>	音声合成の声種 (shimmer ほか)	L109
<code>max_output_tokens</code>	1回の返事の長さの上限	L112
<code>tools</code>	Function Calling 用の関数定義リスト (2日目)	<code>server_function-calling.py</code>

注意 **temperature** は現在のコードには存在しない (API更新で廃止)。探しても見つからないので、時間を使わないこと。

個性を与える ① プロンプトの4層

個性とは「こう喋ってね」という願望ではなく、仕様書である。

(1) 役割

何者なのか。誰のために働くのか

実物の例

あなたは「〇〇薬局」の薬剤師アシスタントです。

(2) 口調・態度

どう喋るか。語尾・一人称・テンション

実物の例

親切・丁寧・正確にお客様の質問に回答します。

(3) 禁止事項

絶対にやらないこと ← 初心者がまず書かない

実物の例

回答は必ず appRAG 関数で得た情報のみに基づき、
自己の知識や推測は使用禁止です。

(4) 逃げ道

答えられないとき、どう振る舞うか ← ここが効く

実物の例

医療相談には自分で判断せず「専門の薬剤師が直接ご説明
しますので、お電話ください」と教えてください。

実験：4つのうち1層だけをわざと削って喋らせ、何が壊れるか観察する。予想 → 実行 → 検証 を紙に書く。

個性を与える ② 探索メニュー Lv.0 – 5

変えたら → 保存 → ブラウザを F5 → 「おはなしをはじめる」。保存だけでは絶対に変わらない。／ 1回に1か所だけ変える。

Lv	何を変える	ファイル / 行	元の値	何が起きるか
0	人格そのもの system_prompt	server_original.py L61-64	薬剤師2行	中身が丸ごと別人になる。すべての出発点
1	声色 voice	server_original.py L109	"shimmer"	alloy / ash / ballad / coral / echo / sage / verse / marin / cedar
2	名前と見た目の文字	pharmacy_agent.html L8 / L749 / L764	MISAKI	画面の名前が自分のAIの名前になる。 ここで初めて「自分のもの」になる
3	会話の間合い silence_duration_ms	server_original.py L105	1000	300 → 食い気味に割り込むせっかちな子 2500 → じっくり考えてから喋る子
4	マイクの敏感さ threshold	server_original.py L104	0.8	下げると小声にも反応する (ただし隣の人の声も拾いやすくなる)
5	話の長さ max_output_tokens	server_original.py L112	1000	100 → 超寡黙 / 4096 → 長広舌 ※料金の上限ではなく1回の返事の上限

Lv.3の「間合い」は、プロンプトでは絶対に作れない個性。数字でしか作れない性格がある。

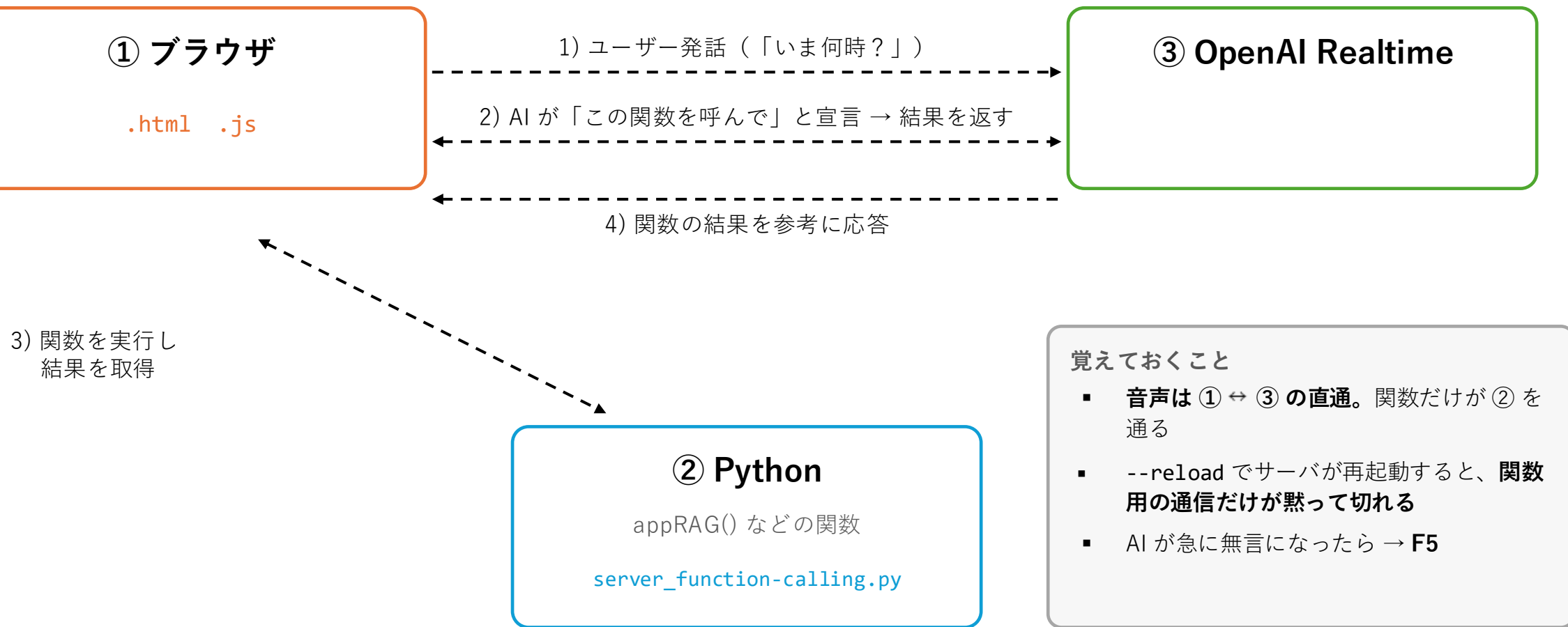
個性を与える ③ 探索メニュー Lv.6 – 11

Lv	何を変える	ファイル / 行	何が起きるか
6	禁止事項と逃げ道で キャラを立てる	server_original.py L61-64 に追記	「～という言葉は絶対に使わない」 「答えられないときは～と言う」。書いた瞬間に性格が安定する
7	入力と出力は別物、 を確かめる	server_original.py L99 language	これは Whisper の聞き取り言語。AI が喋る言語ではない。 "en" にしても AI は日本語のまま。吹き出しだけが英語に化ける
8	アバターの大きさ・位置	pharmacy_agent.js L470	巨大化・移動する。極端にすると画面から消える modelScale: 0.56 / modelX: 200 / modelY: 2000
9	会話中に人格を 差し替える	ブラウザのコンソール (F12)	setPersona("ツンデレの猫。語尾は「にゃ」") F5 せず、会話を続けたまま性格が変わる (voice と model は不可)
10	AI に「できること」を足す Function Calling	server_function-calling.py L233-237 appRAG	返り値を自分の情報に書き換える → 辞書にして 質問ごとに答えを変える → 演習カードへ
11	AI が自分で表情を変える	pharmacy_agent.js L499	setAgentExpression() は実装済みなのに呼ばれていない。 眠っている部品を見つけて活かす、本物の工学体験

アバターを別のキャラに変えるのは1行では無理（専用ライブラリと81個の表情定義に依存）。Lv.2 の名前・Lv.8 の大きさ・CSS の色を触るほうが効果が大きい。

Function Calling

system_prompt が「どう喋るか」なら、関数は「何ができるか」。これも個性のうち。



バックエンド（Function Calling）

Ephemeral Key を取るときのパラメータ

パラメータ	説明
model	利用する会話モデル名
instructions	システムプロンプト
audio.input.transcription	音声 → テキスト変換設定
audio.input.turn_detection	発話検出設定
audio.output.voice	音声合成の声種
max_output_tokens	1回の返事の長さの上限
tools	Function Calling 用の関数定義リスト

↑ ここだけが新しい

関数を実行する WebSocket

/ws/function-call

- 1 リクエスト受信**
呼び出しID・関数名・引数を受け取る
- 2 引数を検証する**
Pydantic が“門番”として型と長さを見る
- 3 関数を実行**
AVAILABLE_FUNCTIONS から名前で引く
- 4 実行結果を送信**
ブラウザ経由で OpenAI に返す

tools のパラメータ

フィールド	説明
type	この定義全体の型。必ず "function"
name	関数の識別名。AI が呼び出すときに使うキー
description	関数は何をするかを AI に伝える説明文
parameters	引数定義をまとめたオブジェクト
$\hookrightarrow$ type	引数定義全体の型。必ず "object"
$\hookrightarrow$ properties	各引数の詳細をプロパティごとに定義
$\hookrightarrow$ search_query	search_query 引数の定義オブジェクト
$\hookrightarrow$ type	その引数の型。ここでは "string"
$\hookrightarrow$ description	その引数は何を表すかの説明
$\hookrightarrow$ required	必須の引数名リスト

3. Function Calling 用ツール定義

```
tool_app_rag = {  
  "type": "function",  
  "name": "appRAG",  
  "description": "Q&Aデータベースを検索し、  
    質問に合致する回答を見つけます。",  
  "parameters": {  
    "type": "object",  
    "properties": {  
      "search_query": {  
        "type": "string",  
        "description": "検索に使う質問文",  
      }  
    },  
    "required": ["search_query"],  
  },  
}
```


フロントエンド (Function Calling)

フロントエンド `pharmacy_agent.js`

UI 状態管理

WebRTC

DataChannel

Function Calling 関連の3イベント

⑤ 関数呼び出し開始

モデルが「関数を呼び出します」と宣言すると呼ばれる。
呼び出しID・関数名・引数（最初は空）を取得できる

⑥ 関数呼び出しの引数チャンク

引数が1文字ずつ送られてくるので、文字列に連結していく

⑦ 関数呼び出し完了

引数の送信が終わったときに呼ばれる。
完全版の引数を WebSocket 経由でバックエンドに渡し、関数を実行する

Function Calling - ① appRAG を辞書にする

最初の import / pip

```
'''❶ 辞書'''  
# 追加のインストールは不要
```

3. Function Calling 用ツール定義

```
'''❶ 辞書'''  
tool_app_rag = {  
    "type": "function",  
    "name": "appRAG",  
    "description": "お店のQ&Aデータベースを検索します。",  
    "parameters": {  
        "type": "object",  
        "properties": {  
            "search_query": {  
                "type": "string",  
                "description": "検索に使う質問文"  
            }  
        },  
        "required": ["search_query"]  
    }  
}
```

4. システムプロンプト

```
'''❶ 辞書'''  
system_prompt = """  
お店のことを聞かれたら必ず appRAG() を呼び出してください。  
"""
```

7. 検索関数（appRAG）と補助ロジック - a

```
'''❶ 辞書'''  
QA = {"営業時間": "午前9時から午後10時までです。",  
      "駐車場": "店舗前に5台分あります。",  
      "処方箋": "受付は閉店30分前までです。"}  
  
def appRAG(search_query: str) -> str:  
    for k, v in QA.items():  
        if k in search_query:  
            return v  
    return "その情報はデータベースにありません。"
```

7. 検索関数（appRAG）と補助ロジック - b

```
'''❶ 辞書'''  
class AppRagArgs(BaseModel):  
    search_query: str
```

Function Calling - ② 時刻

最初の import / pip

```
'''② 時刻'''
import datetime
from typing import Optional
from zoneinfo import ZoneInfo # Windows は pip install tzdata
```

3. Function Calling 用ツール定義

```
'''② 時刻'''
tool_app_rag = {
    "type": "function",
    "name": "appRAG",
    "description": "現在時刻・日付を取得します。"
                  "(location は無視されます) ",
    "parameters": {
        "type": "object",
        "properties": {
            "location": {
                "type": "string",
                "description": "使用しませんが一応受け取ります。"
            }
        },
        "required": []
    }
}
```

4. システムプロンプト

```
'''② 時刻'''
system_prompt = """
「今何時？」など、現在時刻・日付を答える場合は
必ず appRAG() を呼び出してください。
"""
```

7. 検索関数 (appRAG) と補助ロジック - a

```
'''② 時刻'''
def appRAG(location: Optional[str] = None) -> str:
    now = datetime.datetime.now(ZoneInfo("Asia/Tokyo"))
    return now.strftime("%Y-%m-%d %H:%M:%S (%Z)")
```

7. 検索関数 (appRAG) と補助ロジック - b

```
'''② 時刻'''
class AppRagArgs(BaseModel):
    location: Optional[str] = None
```

Function Calling - ③ 天気

最初の import / pip

```
'''Ⓢ 天気'''  
'''pip install requests'''  
import requests  
from typing import Optional
```

3. Function Calling 用ツール定義

```
'''Ⓢ 天気'''  
tool_app_rag = {  
    "type": "function",  
    "name": "appRAG",  
    "description": "指定都市の現在の天気を取得します。  
                  省略時は Tokyo を使います。",  
    "parameters": {  
        "type": "object",  
        "properties": {  
            "location": {  
                "type": "string",  
                "description": "都市名 (例: Tokyo) "  
            }  
        },  
        "required": []  
    }  
}
```

4. システムプロンプト

```
'''Ⓢ 天気'''  
system_prompt = """  
「東京の天気は？」など天気を答える場合は  
必ず appRAG(location) を呼び出してください。  
"""
```

7. 検索関数 (appRAG) と補助ロジック - a

```
'''Ⓢ 天気'''  
def appRAG(location: Optional[str] = None) -> str:  
    city = location or "Tokyo"  
    url = f"https://wttr.in/{city}?format=j1"  
    try:  
        d = requests.get(url, timeout=4).json()  
        c = d["current_condition"][0]  
        return f"{city}は{c['temp_C']}°Cです。"  
    except Exception as e:  
        return f"取得に失敗しました: {e}"
```

7. 検索関数 (appRAG) と補助ロジック - b

```
'''Ⓢ 天気'''  
class AppRagArgs(BaseModel):  
    location: Optional[str] = None
```

Function Calling - ④ 最新ニュース

最初の import / pip

```
'''④ 最新ニュース'''  
'''pip install feedparser'''  
import feedparser
```

3. Function Calling 用ツール定義

```
'''④ 最新ニュース'''  
tool_app_rag = {  
    "type": "function",  
    "name": "appRAG",  
    "description": "Google News 日本語版の  
                    最新3件の見出しを取得します。",  
    "parameters": {  
        "type": "object",  
        "properties": {},  
        "required": []  
    }  
}
```

4. システムプロンプト

```
'''④ 最新ニュース'''  
system_prompt = """  
「最新ニュースは？」など、ニュース見出しを答える場合は  
必ず appRAG() を呼び出してください。  
"""
```

7. 検索関数（appRAG）と補助ロジック - a

```
'''④ 最新ニュース'''  
def appRAG() -> str:  
    rss = "https://news.google.com/rss?hl=ja&gl=JP&ceid=JP:ja"  
    feed = feedparser.parse(rss)  
    if not feed.entries:  
        return "ニュースを取得できませんでした。"  
    top3 = feed.entries[:3]  
    return "\n".join(f"{i+1}. {e.title}"  
                      for i, e in enumerate(top3))
```

7. 検索関数（appRAG）と補助ロジック - b

```
'''④ 最新ニュース'''  
class AppRagArgs(BaseModel):  
    pass    # 引数なし
```

Function Calling - ⑤ AI に表情を選ばせる

pharmacy_agent.js の L499 setAgentExpression() には10表情ぶんの実装があるのに、どこからも呼ばれていない。

① バックエンド：tools に足す

```
tool_emotion = {
  "type": "function",
  "name": "set_emotion",
  "description": "自分の表情を変えます。",
  "parameters": {
    "type": "object",
    "properties": {"emotion": {"type": "string",
      "description": "Joy / Sadness / Anger / Surprised"}},
    "required": ["emotion"]
  }
}
```

② バックエンド：3か所に登録する

```
def set_emotion(emotion: str) -> str:
    return "ok"

class EmotionArgs(BaseModel):
    emotion: str

AVAILABLE_FUNCTIONS["set_emotion"] = set_emotion
FUNCTION_SCHEMAS["set_emotion"] = EmotionArgs
```

③ フロントエンド：受け取って顔を動かす

```
// pharmacy_agent.js 関数の結果を受け取る場所
if (response.name === "set_emotion") {
  Agent.setAgentExpression(args.emotion);
}
```

ここで学べること

- フロント3行では済まない。バックエンドの3か所（tools / AVAILABLE_FUNCTIONS / FUNCTION_SCHEMAS）に登録しないと、AI は無言で固まる
- 他人の書いたコードを読んで、使われていない部品を見つけて活かす —— 実務そのものの体験

用語

API（エーピーアイ）

ソフトウェアやサービス同士が、情報をやりとりするための「つなぎ役」のこと

CDN（シーディーエヌ）

画像やプログラムを世界中に置いておき、近い場所から配る仕組み

DataChannel

音声や映像のほかに、テキストなどのデータをやりとりする専用の通り道

Ephemeral Key

短い時間だけ使える「使い捨ての合鍵」。安全に通信を始めるときに使う

Function Calling

AI が会話の途中で、天気を調べるなどの特別な技（関数）を呼び出して使うこと

GitHub / git clone

プログラムの設計図を置いておく場所と、それを自分のPCに丸ごと複製する命令

HTML / JavaScript

ウェブページの骨組みを作る言語と、そこに「動き」をつける言語

Offer / Answer

通信を始める前に「こんな方法で話そう」とルールを確認し合うこと

PATH（パス）

コマンド名だけ渡された OS が、その実体を探しに行くフォルダの順番リスト

pip install

プログラムを動かすのに必要な「部品（ライブラリ）」を集めてくる命令

Python (.py)

今回のシステムの裏側（バックエンド）で使われているプログラミング言語

requirements.txt

そのプログラムを動かすのに必要な「部品リスト」が書かれたファイル

SDP（エスディーピー）

WebRTC で通信するときに変換される「通信のお約束リスト」

session.update

会話をつないだまま、AI の設定を途中で書き換えるための命令

UI（ユーアイ）

アプリを使うときに目にする画面や、操作するボタンのこと

venv（仮想環境）

このプロジェクト専用の Python の小部屋。壊れたら捨てて作り直せる

WebRTC

ブラウザを通じて、マイクの音声などをリアルタイムにやりとりする技術

WebSocket

サーバと自分のPCの通信路を、つなぎっぱなしにしておく技術

.env（ドットエンフ）

APIキーなど、他人に見せてはいけない設定を書いておく専用のファイル

カレントディレクトリ

ターミナルが「いま自分がいる」と思っているフォルダ

システムプロンプト

AI に与える「役割設定」や「性格」の台本。これで AI の振る舞いが決まる

スーパーリロード

ブラウザが覚えている古いファイルを捨てて読み直すこと（Ctrl + Shift + R）

トークン

AI が文章を理解したり作り出したりするときの、いちばん小さい言葉の単位

バックエンド

アプリの「裏側」で動く部分。目には見えないが、鍵の管理や計算をしている

パラメータ

AI の性格や声の種類、答え方などを細かく調整するための「設定」

フロントエンド

アプリで、みんなが直接見たり触ったりする「表側」の部分

ポート

1台のPCの中の「何番窓口」という番号。今回のサーバは 8000 番

ミュート

マイクを一時的に切ること。ただし通信自体は続いているので、使い終わったらタブを閉じる

困ったときは

症状	原因	どうする
command not found: py	PATH に載っていない	「Add python.exe to PATH」にチェックを入れて入れ直す
ModuleNotFoundError	venv に入っていない／pip 未実行	(venv) を確認 → <code>py -m pip install -r requirements.txt</code>
Got unexpected extra argument	ハイフンが半角2つになっていない	--reload を手で打ち直す
(venv) が出ない (Windows)	PowerShell の実行ポリシー	venv\Scripts\activate.bat を使う
Address already in use	前のサーバが生きている	Ctrl+C で全部止めて、1つだけ起動し直す
401 Incorrect API key ... None	.env が読めていない	.env.txt になっていないか確認。VSCode から作り直す
「準備しています…」から進まない	アバターの画像が読めていない	F12 → Network タブで赤い行のドメインを読む
ボタンを押したら画面が戻った	ネットワークが通信を通していない	別の回線（テザリング）を試す
「マイクが使用できません」	Firefox を使っている	Chrome を使う 。これは絶対条件
マイクが取れない・音が入らない	file:// で開いている／OS側で未許可	Live Server で開く。設定 → プライバシー → マイク も許可する
喋っても AI が反応しない	threshold が高い／声が小さい	L104 を 0.5～0.6 に。まずイヤホンマイクを口元へ
言い終わる前に AI が喋り出す	silence_duration_ms が短い	L105 を 1800～2500 に
直したのに変わらない	F5 していない／保存していない	設定は接続した瞬間に一度だけ送られる。必ず F5
AI が急に無言になった	関数用の通信だけが切れた	ブラウザを F5 して接続し直す
何を聞いても「営業時間は…」	仕様通り 。appRAG がダメー	壊れていない。AI が嘘をつかないよう縛れている証拠

まず最後の行を読む。ファイル名と行番号が書いてあれば、そこを開く。それだけで9割は解決する。